Введение в dbt. Домашнее задание

Задание

Возьмите датасет из домашки по ClickHouse (таблица `transactions_db.transactions` с данными о транзакциях и признаком фрода) и реализуйте на его основе проект dbt.

1) Модель данных: от raw до витрин

Создайте слоистую архитектуру моделей (если хотите, то слоёв мб больше):

```
raw (source) → staging → marts
```

Staging-слой

- `stg_transactions` — очистка и нормализация сырых данных, приведение типов, добавление вычисляемых полей (например, сегментация суммы через макрос)

Витрины (marts)

Реализуйте следующие аналитические витрины (все не обязательно – от количества реализованных зависит оценка – см в самом конце таблицу оценок):

1. `mart_daily_state_metrics` — Дневные метрики по штатам

- Агрегация транзакций по дате и штату

- Метрики: количество транзакций, сумма, средний чек, P95, доля крупных транзакций

2. `mart_fraud_by_category` — Анализ фрода по категориям

- Выявление категорий с наибольшим уровнем мошенничества

- Метрики: общее число транзакций, число фродов, fraud_rate (%), суммы

3. `mart_fraud_by_state` — Географический анализ фрода

- Распределение фрода по штатам США

- Метрики: fraud_rate, уникальные клиенты/мерчанты, суммы

4. `mart_customer_risk_profile` — Профиль риска клиентов

- Сегментация клиентов по уровню риска (HIGH/MEDIUM/LOW)

- История транзакций, fraud_rate на клиента, средний чек

5. `mart_hourly_fraud_pattern` — Временные паттерны фрода

- Анализ по дням недели и часам

- Выявление временных окон с повышенным риском

6. `mart_merchant_analytics` — Аналитика по мерчантам

- Метрики по каждому мерчанту: оборот, fraud_rate, флаг подозрительности

2) Описания, тесты и макросы

Макросы

- Создайте минимум 1 собственный макрос (например, `amount_bucket` для сегментации сумм)

- Используйте макрос в staging или mart моделях

Метаданные (schema.yml)

- Опишите все модели и ключевые колонки

- Добавьте descriptions для бизнес-контекста

Тесты

- Базовые: `not_null`, `unique`, `accepted_values`

- На витрины: проверка уникальности комбинаций ключей

3) Документация

Сгенерируйте dbt Docs:

```bash
```

dbt docs generate

dbt docs serve

```

Приложите скриншот DAG или страницы документации.

4) Дополнительные артефакты (по желаемой оценке)

На 6-7 баллов:

- ✓ Создать и использовать свой макрос

- ✓ Использовать стандартные пакеты: `dbt_utils`, `dbt_date`

- ✓ `schema.yml` с описанием метаданных и тестами

На 8-9 баллов:

Всё из 6-7, плюс:

- ✓ Использовать любой из пакетов: `dbt_expectations`, `elementary`, `dbt_project_evaluator` (или все)

- ✓ Добавить singular tests (кастомные SQL-тесты в папке `tests/`)

Примеры singular tests:

```sql

-- tests/assert_no_negative_amounts.sql

SELECT * FROM {{ ref('stg_transactions') }} WHERE amount < 0

-- tests/assert_fraud_rate_bounds.sql

SELECT * FROM {{ ref('mart_fraud_by_category') }} WHERE fraud_rate > 100 OR fraud_rate < 0

```

На 10 баллов:

Всё из 8-9, плюс:

- ✓ Добавить unit tests (dbt 1.8+)

- ✓ Настроить `sqlfluff` для линтинга SQL
```

- ✅ Добавить `.pre-commit-config.yaml`

- ✅ Добавить `Makefile`

Пример `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: https://github.com/sqlfluff/sqlfluff
    rev: 3.0.0
    hooks:
      - id: sqlfluff-lint
        args: [--dialect, clickhouse]
      - id: sqlfluff-fix
        args: [--dialect, clickhouse]
```

Структура проекта (пример)

```
dbt/
├── dbt_project.yml
├── packages.yml
├── profiles.yml (или ~/.dbt/profiles.yml)
├── models/
│   ├── sources/
│   │   └── sources.yml
│   ├── staging/
│   │   ├── stg_transactions.sql
│   │   └── stg_transactions.yml
│   └── marts/
│       ├── mart_daily_state_metrics.sql
│       ├── mart_fraud_by_category.sql
```

```
|       ├── mart_fraud_by_state.sql
|       ├── mart_customer_risk_profile.sql
|       ├── mart_hourly_fraud_pattern.sql
|       ├── mart_merchant_analytics.sql
|       └── schema.yml
├── macros/
|   └── amount_bucket.sql
├── tests/                  # singular tests (8-9)
|   ├── assert_no_negative_amounts.sql
|   └── assert_fraud_rate_bounds.sql
├── seeds/
|   └── states.csv
├── .sqlfluff               # (на 10)
└── .pre-commit-config.yaml   # (на 10)
```

Схема данных (source)

Таблица `transactions_db.transactions`:

Колонка	Тип	Описание
transaction_time	DateTime	Время транзакции
merch	String	Мерчант
cat_id	String	Категория (14 значений)
amount	Float64	Сумма
name_1, name_2	String	Имя, фамилия клиента
gender	String	Пол (M/F)
us_state	String	Штат США (50 штатов)
lat, lon	Float64	Координаты клиента

| merchant_lat, merchant_lon | Float64 | Координаты мерчанта |

| target | UInt8 | Признак фрода (0/1) |

Команды для запуска

```bash
cd dbt/

# Установка зависимостей
dbt deps

# Загрузка seeds
dbt seed

# Запуск моделей
dbt run

# Запуск тестов
dbt test

# Генерация документации
dbt docs generate && dbt docs serve

# Полный цикл
dbt deps && dbt seed && dbt run && dbt test
```

Что сдать

1. PR с каталогом `dbt/` и всеми файлами

2. В описании PR:

- Версии `dbt-core` и `dbt-clickhouse`
- Скриншот/лог успешного `dbt test`
- Скриншот DAG из dbt Docs
- Список реализованных витрин и тестов
- Для 10 баллов: конфиг sqlfluff и pre-commit

Критерии оценки

Баллы	Кол-во витрин для реализации	Требования
6-7	2	Макрос + пакеты (dbt_utils, dbt_date) + schema.yml с тестами
8-9	4	+ dbt_expectations/elementary/dbt_project_evaluator + singular tests
10	6	+ unit tests + sqlfluff + .pre-commit-config.yaml + Makefile